



# Step-by-step procedure to create your own *customized* version of the platform

Step by step procedure, in 7 languages, without writing code.

A guide by Pessoa Academy · Vibe Coding Workflow™

---

10

STEPS

7

LANGUAGES READY

0

LINES OF CODE WRITTEN AVERAGE TIME  
BY HAND

~2 h

## You are the *director*, the AI is the typist

The whole guide rests on one idea: you describe what you want in plain language, the AI assistant writes the code in your editor. Whenever you see a PROMPT section, copy the sentence and paste it into the AI assistant chat inside Visual Studio Code.

## Set up your *toolbox*

**WHY** Before writing anything, you need a handful of free tools that have become standard in software work. They're not random apps — each has one specific job and they're designed to plug into one another. Think of a workshop: **GitHub** is the storeroom, **Visual Studio Code** is the workbench where you open files, the **AI assistant** is your apprentice who writes code on your dictation, **GitHub Desktop** is the trolley moving stuff between storeroom and workbench, **Supabase** is the data warehouse, **Netlify** is the spotlight that puts the finished product in the shop window. Today is sign-ups and installs only — no code yet.

### WHAT YOU DO

1. Go to **github.com** and create a free account (an email address is enough). GitHub is the diary where every change to your code is timestamped and saved: if you mess up, you rewind to yesterday afternoon in one click. From here on it's the anchor everything else hangs from.
2. Download **Visual Studio Code** from [code.visualstudio.com](https://code.visualstudio.com) (Windows/Mac/Linux, ~100 MB). It's the program where you open, read and edit your project files — think of it as Word for code. On first launch it looks like a plain text editor: the magic kicks in at the next step, when you bolt the AI on top of it.
3. In VS Code click the *Extensions* icon in the left sidebar (the four-square icon, shortcut Ctrl+Shift+X on Windows/Linux, Cmd+Shift+X on Mac). Search for a free AI assistant: the easiest is **Gemini Code Assist** by Google, but **Codeium**, **Continue** or **Cody** all work fine. Install it and sign in with a Google account: from this point on you have a chat panel inside VS Code where you can give instructions in plain English.
4. Download **GitHub Desktop** from [desktop.github.com](https://desktop.github.com) and sign in with the same account from step 1. It's a small app that bridges your computer and GitHub: with two clicks your local changes fly to the cloud (commit + push). Without it, you'd have to learn Git's command line — we can do without that.
5. Go to **supabase.com** and sign up (Google or GitHub login is the fastest way in). Supabase is the online warehouse where your app will store everything: users, participants, mobility records, uploaded documents. For now you just need the account ready — the actual project comes at step 03, when we need its credentials.
6. Go to **netlify.com** and sign up — "Sign up with GitHub" is the quickest path, since the two services then talk to each other automatically. Netlify is what turns your code into a website anyone on the internet can reach, for free. No server to set up, no FTP, nothing: it'll rebuild and republish every time you push to GitHub.
7. Download **Node.js**, the LTS (Long Term Support) version, from [nodejs.org](https://nodejs.org). Node.js is the engine that runs the site on your computer while you're testing it: without it, the `npm run dev` command at step 05 wouldn't know what to execute. The installer is wizard-style — Next, Next, Done.

---

**DON'T WORRY ABOUT COST** All of these tools are free for personal use: nobody asks for a credit card to get started. If at some point you're asked to pay — stop and double-check: you're about to upgrade to a paid tier by

mistake, and nothing in this guide ever requires it.

---

## 02 FORK & CLONE

# Make your *personal copy* of the project

**WHY** "Forking" means making a personal copy of the Managemob project on GitHub: you work on your copy, the original stays untouched. "Cloning" means downloading that copy onto your own computer so you can open it in VS Code. Two clicks and one command — by the end of this step you have the actual project, on your disk, ready to run.

### WHAT YOU DO

1. Go to the GitHub page of the Managemob template — e.g. [github.com/ETN-International/managemob-app](https://github.com/ETN-International/managemob-app), or whatever template URL you've been given. Top right, click **Fork**: GitHub asks which account should own the copy (yours) and what to call it. Pick something short and lowercase, like *peessoa-mobility*. A few seconds later, the copy is yours.
2. On your new fork's page, click the green <> **Code** button (top right). A menu opens — choose **Open with GitHub Desktop**. Your browser will ask permission to launch the app — allow it.
3. GitHub Desktop suggests a local folder to save the project. Either accept the default or pick a convenient path (e.g. *Documents* or *Desktop/Projects*) and click **Clone**. In a few seconds, every project file lives on your computer.
4. Top right in GitHub Desktop, click **Open in Visual Studio Code**: VS Code opens with the right folder already loaded. On the left, the *Explorer* panel shows the full project structure.
5. Open the AI assistant chat in VS Code and paste this prompt: `Install the project dependencies for me`. The AI will download all the libraries the project depends on (a couple of minutes). When it tells you it's done with no errors, you're ready for the next step.

### PROMPT TO PASTE INTO THE AI ASSISTANT

"In one paragraph, walk me through what the `src/` folder contains — what the main subfolders are and what each one is for. Don't change anything yet, I just want a map of the project."

## Prepare the *data warehouse*

**WHY** Supabase is the online warehouse where the app stores everything: users, participants, mobility records, uploaded documents. In this step you create your Supabase space, grab the two credentials (URL + key) the site needs to talk to it, and create your first administrator user. The rest — filling the database with tables, locking down security — the AI does for you in the next step. You just point and click.

### WHAT YOU DO

1. Go to [supabase.com](https://supabase.com), sign in (or sign up if it's your first time), and click **New project**. Pick a name (e.g. *pessoa-mobility*), a **European region** (Frankfurt, Ireland or Stockholm — closer to your users = snappier app) and a strong password. **Save the password** somewhere safe: you can't recover it later, and you may need it for advanced operations.
2. Wait 1–2 minutes. Supabase is provisioning a dedicated workspace for you. The dashboard might be unresponsive for a few seconds — that's normal, it's not stuck.
3. In the left sidebar, click **Project Settings** → **API**. You'll see two values written in plain text: **Project URL** (an address starting with *https://*) and **anon public key** (a long string). Copy both into a temporary scratch note: you'll need them in a moment, at step 04. Nothing technical — just two copy-pastes.
4. Still in the sidebar, go to **Authentication** → **Providers**. Make sure **Email** is enabled (usually it is by default). For testing, disable *Confirm email*: that way, when you create your user in the next action, you won't have to check your inbox to click a confirmation link. You'll re-enable it before going to real production (see step 10).
5. Sidebar → **Authentication** → **Users** → **Add user** → **Create new user**. Create your first administrator: an email and a password you'll remember. This is the user you'll log in with at step 05 to test the site.

# Let the *AI assistant* wire it all together

**WHY** The site now knows there's a Supabase warehouse, but it doesn't have the credentials yet — and the warehouse is empty. In this step the AI does three technical things for you in one go: it saves your credentials in the right place, fills the database with the app's tables, and turns on security. You paste a single prompt and say "yes" when it asks for confirmation.

### WHAT YOU DO

1. Open the AI assistant chat inside Visual Studio Code (in Gemini Code Assist it's the star icon in the sidebar; in Cursor press `Ctrl+L` / `Cmd+L`). If you see an option like *Agent mode* or *@workspace*, turn it on: that means the AI is allowed to read and edit project files.
2. Copy this whole prompt (it's already written — you don't need to understand every word) and paste it into the AI chat:
3. Configure the Managemob project with these Supabase credentials:  
URL = `https://<your-project>.supabase.co`  
KEY = `<your anon key>`

Please do three things: (1) create the `.env` file with these two variables (`VITE_SUPABASE_URL` and `VITE_SUPABASE_ANON_KEY`) and make sure `.env` is in `.gitignore` so it never ends up on GitHub; (2) run the file `supabase/schema.sql` against my Supabase database to create the app's tables; (3) enable Row Level Security on every table, so only logged-in users can read and write. Show me each step before you apply it and ask for confirmation.

4. Replace `https://<your-project>.supabase.co` and `<your anon key>` with the two values you copied from the Supabase dashboard at step 03. Hit Enter.
5. The AI will walk you through each operation: "I'm creating the `.env` file, ok?", "I'm running this SQL, ok?", "I'm turning on security, ok?". To each prompt, answer **yes** (or accept the diff). When it tells you it's done, you're set: the site is connected to Supabase, the database has its tables, security is on — and you didn't touch a single file by hand.

---

**IF THE AI ASKS FOR SOMETHING YOU DON'T UNDERSTAND** It's totally fine to say "explain what you're about to do, in plain English, before doing it". That's its job — you don't need to learn SQL or environment variables to finish this guide. If something feels wrong (e.g. it wants to delete files that aren't related), say "stop, I didn't approve that" and try again with more specific instructions.

---

# Run the site *locally*

**WHY** Before pushing the site online, it pays to run it on your own machine first. This is when you find out whether Supabase is wired up correctly, whether login works, whether data really gets saved. If something is off you catch it here — without anyone else seeing it.

### WHAT YOU DO

1. In VS Code, open the AI assistant chat and paste: `Start the site locally so I can preview it in the browser.` The AI will run the right command for you (it's `npm run dev` if you're curious). Prefer doing it by hand? Open the Terminal (*View* → *Terminal*), type `npm run dev`, hit Enter.
2. In the terminal, you'll see a line like `Local: http://localhost:XXXX`. Open that address in your browser (usually `http://localhost:3000`; if that port is busy, Vite picks another one and shows you which). If the AI started it for you, it'll tell you the address right in the chat.
3. You'll see the login screen. Sign in with the email and password of the administrator you created at step 03. If it shoots back "Invalid login credentials", just ask the AI: `Login isn't working, please check the Supabase credentials.` Nine times out of ten it's a copy-paste glitch or a stray space.
4. Once inside, try creating a test participant from the home page (just name, surname, destination country). Save, then press F5 to reload: if the participant is still there, your live database is genuinely persisting data.
5. Click the language switcher (top of the page, both on login and in the sidebar): the interface should switch instantly across all 7 languages. If a translation is missing or something looks off, you'll spot it here.

### IF SOMETHING FAILS, USEFUL PROMPT

"If something goes wrong when I start the site, tell me what the terminal says and propose a simple fix. Don't change any file without asking me first."

## Personalize across *7 languages*

**WHY** Managemob speaks 7 languages out of the box. You don't translate from scratch — you just need to rewrite the user-facing strings (labels, titles, button text) so they fit YOUR organisation, and naturally in each language. The AI does this with a single prompt: you pick the tone, the AI applies it across all 7 files at once.

### WHAT YOU DO

1. In VS Code, open the AI assistant chat. You don't have to open the 7 locale files one by one — the AI handles all that.
2. Paste this prompt (replacing "Pessoa Academy" with your organisation's name and tweaking the tone description):
3. Rewrite every user-facing string in the app for the brand 'Pessoa Academy', with a warm and professional tone (change this if you want a different style). Open all 7 files inside `src/locales/` (`it.ts`, `en.ts`, `es.ts`, `fr.ts`, `de.ts`, `pt.ts`, `se.ts`), edit only the values (the strings between quotes), don't touch the keys (the names before the colon). Translate naturally in each language, not word for word. Show me the diff before applying.
4. The AI proposes every change in a single review. Scroll through the diff: if you like the wording, accept. If you want a different tone (more formal, less corporate, friendlier), just tell it — for example "redo it more formal, addressing the user as 'Sir/Madam'" — and it'll redo the lot.

---

**ADDING AN 8TH LANGUAGE** Want one more language beyond the 7 (e.g. Polish)? Ask the AI: *"Add Polish as the 8th app language by copying en.ts and translating. Add the flag in the language switcher too."* It'll do it in 30 seconds, no code touching from your side.

---



# Give it *your visual identity*

**WHY** Looks are the first thing your users notice. Once you swap the logo, the colours and the page title, Managemob stops being called Managemob and visibly becomes YOUR platform. These are purely cosmetic changes — the logic stays the same — but they completely change the feel.

### WHAT YOU DO

1. Prepare your logo as a `logo.svg` file (preferred — it's a vector, scales perfectly) or `logo.png` (square, at least 256×256 pixels, transparent background). Drag the file into the project's `public/` folder in VS Code: you'll see the new icon appear in the Explorer.
2. Pick your two colours: **primary** (the app's main colour, e.g. your organisation's blue) and **accent** (a complementary colour for details and buttons). If you don't have a palette ready, head over to [coolers.co](https://coolers.co), play around until you like it, and note the two hex codes (e.g. `#1D72B8` and `#38BDF8`).
3. Open the AI chat and paste this prompt (replacing the org name and colour codes):
4. Full rebrand of the app for my organisation 'Pessoa Academy'. (1) Replace the 'M' placeholder logo with the file `public/logo.svg` in `Sidebar.tsx` and `Login.tsx`. (2) Update the colours in `src/styles/main.css`: `--primary = #XXXXXX`, `--accent = #YYYYYY`. (3) Change the page title (browser tab) to 'Pessoa Academy' in `index.html`, and point the favicon to my new logo. (4) Replace the word 'Managemob' with 'Pessoa Academy' in the user-facing strings across all 7 files in `src/locales/`. Show me the diff before applying each change.
5. The AI proposes all the changes. Approve. Reload the browser on `localhost:3000`: your visual identity is already showing.

## Automate your *documents*

**WHY** One of the most powerful things Managemob does: take any Word document (Learning Agreement, Europass Mobility, mobility contract...) and turn it into a template that auto-fills with the data of whichever participant you select. End of manual copy-paste: 50 participants, 50 documents, 30 seconds.

### WHAT YOU DO

1. Open the Word document you want to automate, in Microsoft Word, LibreOffice or Google Docs. If it's currently a PDF or a scan, you'll need to recreate it as **.docx** first — the app only accepts **.docx** files.
2. Find the spots where variable data goes (participant name, dates, host company, country...) and replace them with curly-brace tags. Practical examples:  
[First Name] [Surname] → {name} {surname}  
[Start date] → {arrival\_date}  
[Email] → {email}  
[Host company] → {host\_company\_name}. The full list of available tags lives on the *Documents* page of the app, behind "Export Placeholders".
3. Save as **.docx** (not **.doc**, not **.pdf**, not Google Docs). In Word: *Save as* → *Word Document (\*.docx)*.
4. In Managemob: *Documents* → *Upload Template*. Give the template a name (e.g. "Learning Agreement EN"), pick the **.docx** file, confirm. The template appears in your list.
5. Select it, search and tick one or more participants in the list, click **Generate**. One participant gets you a single file; multiple participants get you a ZIP containing one document per person. Formatting (bold, italics, tables) is preserved: the app only swaps the tags.

---

**TAG RULES THAT MATTER** Tags are **case-sensitive**: write {name}, not {Name}. No spaces inside the braces ({ name } won't work). Unknown tags stay in the document as literal text ("{wrongName}"): if your generated document still has braces in it, you mistyped a field name. For the exact list, hit *Documents* → *Export Placeholders (CSV)*.

---

## Publish the site *on Netlify*

**WHY** So far the site only runs on your own machine, where nobody else can see it. To share it with the world, you push it onto Netlify — free, hooked straight into your GitHub repository. From this point on, every push to GitHub triggers Netlify to rebuild and redeploy automatically: no FTP, no deploy commands, no nothing.

### A · Save your changes to GitHub (with GitHub Desktop)

1. Open GitHub Desktop. If you've modified any files (and after steps 06–07 you've modified plenty), the top-left panel shows the list of changed files with a colour-coded diff (green = added, red = removed).
2. Bottom-left there's a save-message field: write a short description of what you did (e.g. "Pessoa Academy customisation: logo, colours, copy"). Click **Commit to main**: you've sealed a save point.
3. Top of the window, click **Push origin**. Your changes have flown up to your GitHub repository — finally backed up, finally ready for Netlify.

### B · Publish on Netlify (linked to your repository)

1. Go to [netlify.com](https://netlify.com) and sign in (the quickest way is *Sign up with GitHub*). Dashboard → **Add new site** → **Import an existing project** → **Deploy with GitHub**.
2. First time round, Netlify will ask permission to read your repositories — grant it. Then pick the **fork** you created at step 02 from the list.
3. Netlify auto-detects this is a Vite project and proposes the right build settings. Just verify they read: *Build command*: `npm run build` · *Publish directory*: `dist` · *Branch*: `main`.
4. Click **Show advanced** → **Add environment variable**. Add the TWO variables the AI put in your `.env` file at step 04: `VITE_SUPABASE_URL` and `VITE_SUPABASE_ANON_KEY` (same values). Without these, the deployed app won't know how to reach Supabase.
5. Click **Deploy**. The first build runs in 1–3 minutes — you can watch the log scroll live. When it says "Site is live", you're done.
6. Netlify hands you a random address like `https://<funny-words-123>.netlify.app`. **That's your live site!** From *Site settings* → *Change site name* you can rename it (e.g. `pessoa-mobility.netlify.app`); from *Domain management* you can hook up your own domain if you have one (e.g. `mobility.pessoa-academy.com`).

#### PROMPT TO PASTE INTO THE AI ASSISTANT

"Add the configuration to the project that makes every internal page work correctly when the site is online on Netlify (so that if I refresh a page like /dashboard I don't get a 404 Not Found error). Show me the file before creating it."

## You're *online*, last verification

**WHY** You're online. Before opening the doors to real users, let's check that everything works from the public Netlify URL (not localhost any more). It's 7 quick checks: pass them all and you ship. Fail any of them and you've still got time to fix it — without anyone noticing.

---

**BEFORE THE OFFICIAL GO-LIVE** When you transition from internal beta to real users, do this cleanup in Supabase: (1) **re-enable Confirm email** in Authentication → Providers (so new sign-ups have to verify their email); (2) **rotate the DB password** in Project Settings → Database (replace the password you saved at step 03); (3) delete the **test administrator** and create the real ones; (4) take a **backup** of the database (Project Settings → Database → Backups). Twenty minutes of work that'll save your bacon a hundred times over.

---

### FINAL CHECKLIST

- Logging in from the public Netlify URL works (with the administrator created at step 03).
- The language switcher shows all 7 languages and the text actually changes when I click them.
- I add a test participant, hit F5: if it's still there, the live database is persisting data.
- I generate a Word document for one participant: the {name}-style tags in the downloaded file have been replaced by real values.
- The Calendar shows the participant on the correct dates.
- Refreshing a sub-page like /dashboard doesn't 404 (= the prompt from step 09 has been applied).
- Logging out lands me back on the login screen with no errors.

---

## A platform *that's yours*, in 7 languages, no code written by hand

Visual Studio Code opens the files. The AI assistant writes the code. GitHub Desktop syncs everything online. Supabase stores the data. Netlify puts the site in the shop window. You orchestrate the whole thing and own the decisions that matter: name, colours, copy, automated documents, languages. That's the Vibe Coding Workflow™: zero lines of code written by hand, but the result is a platform that's 100% yours.

Vibe Coding · Pessoa Academy · Managemob · VS Code · Gemini Code Assist · GitHub Desktop · Supabase · Netlify · Erasmus+ · Learning Mobility